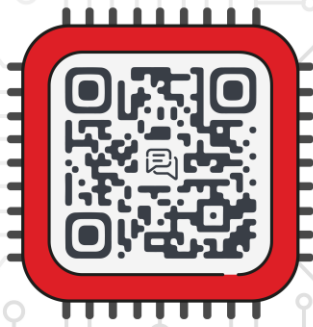




1

26

3900 sp

عملي

كلية الهندسة المعلوماتية

السنة الرابعة - الذكاء الصناعي

Introduction to python

عملي مشترك



محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 30/10/2025

الشبكات العصبونية

• لمحة سريعة عن اللغة:

1. لغة بايثون هي لغة سهلة القراءة والتعلم.
2. تشبه لغة البشر.
3. هي لغة مفتوحة المصدر.
4. متعددة النماذج يمكن الكتابة بنموذج إجرائي أو وظيفي أو كائن التوجه (OOP).
5. تعمل لغة بايثون على نظام مفسر (Interpreter System) وهذا يمكننا من كتابة البرامج بشكل تفاعلي وتنفيذ الكود بمجرد كتابته

• آلية العمل (فائدة):

1. التحويل إلى Byte Code : عند تشغيل ملف بايثون الذي لاحقته من النوع (.py) ، يقوم المفسر أولاً بتحويل الكود المصدري إلى تنسيق بسيط عالي المستوى يسمى Byte Code .
2. يتم تخزين هذا البايت كود في ملفات لاحقتها (.pyc).
3. البايت كود ليس كود آلة، وإنما هو مجموعة من التعليمات التي يسهل على الآلة الافتراضية فهمها.
4. التنفيذ بواسطة الآلة الافتراضية (PVM) : يتم تشغيل وتنفيذ هذا البايت كود بواسطة محرك خاص يُعرف باسم Python Virtual Machine آلة بايثون الافتراضية.
5. تقوم الـ PVM بقراءة تعليمات البايت كود وتنفيذها سطرًا بسطر على النظام الفعلي.

❗ باختصار: يقال أنّ بايثون مفسرة لأنها لا تتحول مباشرة إلى كود آلة يعمل على المعالج بشكل مباشر بل تعتمد على محرك بسيط PVM لتنفيذ تعليماتها بالوقت الفعلي .



استخدامات لغة بايثون

- تستخدم بكثرة في المجالات العلمية.
- خاصة في مجالي التعلم الآلي (Machine Learning) وعلوم البيانات (Data Science) .
- وذلك بسبب أدائها العالي ومكتباتها الممتازة للحسابات العددية (numerical calculations) والمعالجة الإحصائية (statistical processing) مثل (Numpy and Scipy)
- تحتل مكانة قوية كركيزة أساسية في أطر التعلم العميق (Deep Learning Frameworks) .
- تتيح للمطورين كتابة برامج بأقل عدد من الأسطر مقارنة ببعض لغات البرمجة الأخرى.

المكتبات الخارجية

هدفنا الرئيسي هو بناء الشبكات العصبونية من الصفر أي أن نبدأ بتصميمها خطوة بخطوة ولهذا السبب سياستنا هي ألا نستخدم الكثير من المكتبات الجاهزة .

لكن هناك استثناءان ضروريان سنعتمد عليهما وهما: NumPy و Matplotlib .

سنستخدم هاتين المكتبتين لأنهما تساعداننا على تنفيذ عملية التعلم العميق (Deep Learning) بشكل فعال وسريع .

⚡ NumPy (Numerical Python) :

هي مكتبة للحسابات العددية .
توفر العديد من الطرق المريحة للتعامل مع الخوارزميات والمصفوفات الرياضية المتقدمة

⚡ Matplotlib :

مكتبة لرسم المخططات والرسوم البيانية (Plots and Graphs) .
تستخدم لعرض النتائج التجريبية والتحقق بصرياً من البيانات أثناء تنفيذ الشبكة العصبونية.

مفهوم المسافة البادئة (Indentation) :

تستخدم بايثون المسافة البادئة للإشارة إلى كتلة من الكود (indicate a block of code) .
يجب استخدام نفس عدد المسافات في نفس كتلة التعليمات البرمجية وإلا سيظهر خطأ (error) .

⚡ أمثلة :

```
1
2 if 5 > 2:
3     print("Five is greater than two!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Five is greater than two!

المثال السابق يمثل الحالة الصحيحة للبادئة .

```
2 if 5 > 2:
3     print("Five is greater than two!")
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

File "c:\vs projects\python projects\test.py", line 3

```
print("Five is greater than two!")
```

IndentationError: expected an indented block after 'if' statement on line 2

```
2 if 5 > 2:
3     print("Five is greater than two!")
4     print("Five is greater than two!")
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

File "c:\vs projects\python projects\test.py", line 4

```
print("Five is greater than two!")
```

IndentationError: unexpected indent

المثالان السابقان يوضحان الحالة الخاطئة للبادئة .

أساسيات بايثون

• التعليقات (Comments) :

تستخدم بايثون الرمز # لتعليق الأسطر التي لا تريد تنفيذها.

• تعليمة الطباعة print() :

- تستخدم لطباعة أي بيانات .
- يمكن طباعة عدة أشياء بجانب بعض باستخدام الفاصلة (,) .
- لا يمكن كتابة عدة تعليمات print بنفس السطر دون استخدام فاصلة منقوطة ; للفصل بينهما وإلا سيؤدي ذلك إلى خطأ (error)

أمثلة :

```
1
2 print("Python is fun!") print("Really!")
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

File "c:\vs projects\python projects\test.py", line 2

```
print("Python is fun!") print("Really!")
```

SyntaxError: invalid syntax. Did you mean 'in'?

الكود السابق يمثل الحالة الخاطئة عند استعمال عدة تعليمات طباعة بنفس السطر .

```
2 print(5,10) ; print("Really!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
5 10
Really!
```

الكود السابق يمثل الحالة الصحيحة وهي تتم بوضع فاصلة منقوطة بين تعليمتين الطباعة في نفس السطر

- لا يمكن كتابة string بداخل تعليمة الطباعة بدون علامتي تنصيص " لأن المفسر لا يمكنه تفسير ما معنى الكلام ، وهذا يعطي خطأ (error) . ولكن يمكننا كتابة أرقام وسيفهمها

🔗 مثال :

```
2 print(Hello world)
```

PROBLEMS **2** OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
File "c:\vs projects\python projects\test.py", line 2
print(Hello world)
      ^^^^^^^^^^^
```

SyntaxError: invalid syntax. Perhaps you forgot a comma?

- يمكن استخدام الرمز + بدلاً من الفاصلة (,) للفصل بين مدخلات التعليمة ولكن عند جمع int مع string باستخدام + يعطي خطأ (error) ، ويفضل في هذه الحالة استخدام الفاصلة (,) .

🔗 مثال :

```
2 print("Hello world" + 5)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Traceback (most recent call last):
  File "c:\vs projects\python projects\test.py", line 2, in <module>
    print("Hello world" + 5)
    ~~~~~
```

TypeError: can only concatenate str (not "int") to str

- تعليمية الطباعة print تنتقل لسطر جديد بشكل افتراضي (\n ضمناً) ، ولجعل الخرج في سطر واحد عند كتابة عدة تعليمات print، يتم استخدام الـ end keyword داخل التعليمة

🔗 مثال :

```
2 print("Hello world" , end= " ")
3 print(5)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Hello world 5
```

• العمليات الرياضية:



- يمكن استخدام العمليات الرياضية الأساسية +, -, *, /.
- الرمز ** يشير إلى عملية الأس.
- الرمز // يعطي قيمة int من ناتج القسمة (القسمة الصحيحة).
- ناتج عملية القسمة (/) دائماً يكون على شكل عشري (float) حتى لو كان الناتج عدداً صحيحاً.

↪ مثال :

```

-
2  x = 5 ; y=3
3  result1 = x + y
4  result2 = x - y
5  result3 = x * y
6  result4 = x / y
7  result5 = x ** y
8  result6 = x // y
9  result7 = x % y
10 print("Result1 = " , result1 , end=" | ")
11 print("Result2 = " , result2 , end=" | ")
12 print("Result3 = " , result3 , end=" | ")
13 print("Result4 = " , result4 )
14 print("Result5 = " , result5 , end=" | ")
15 print("Result6 = " , result6 , end=" | ")
16 print("Result7 = " , result7 , end=" | ")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Result1 = 8 | Result2 = 2 | Result3 = 15 | Result4 = 1.6666666666666667
Result5 = 125 | Result6 = 1 | Result7 = 2 |

• أنواع البيانات (Data Types) :

- Int : عدد صحيح موجب أو سالب.
- Float : عدد عشري.
- bool : True أو 1 و False أو 0 .
- string : سلسلة نصية .

• المتغيرات (Variables) : القيمة = اسم المتغير

- التحديد الديناميكي للنوع (Dynamic Type Conversion) : لا داعي لتحديد نوع المتغير لأن لغة بايثون تتمتع بتحديد النوع الديناميكي، حيث تعرف النوع تلقائياً بمجرد وضع القيمة.
- يمكن استخدام التابع type() لتحديد نوع البيانات أو نوع المتغيرات.
- تعيين القيم:
- يمكن إعطاء قيمة لأكثر من متغير في نفس السطر.
- يمكن إعطاء قيمة واحدة لأكثر من متغير.

```
2 x = "Hi" ; y=10/3
3 print(type(5))
4 print("Type of x is : " , type(x))
5 print("Type of y is : " , type(y))
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
<class 'int'>
Type of x is : <class 'str'>
Type of y is : <class 'float'>
```

- اعطاء عدة قيم لعدة متغيرات في نفس السطر (Many values to multiple variables):

```
2 x, y, z = "Orange", "Banana", "Cherry"
3 print(x)
4 print(y)
5 print(z)
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Orange
Banana
Cherry
```

- اعطاء قيمة واحدة لعدة متغيرات (One value to multiple variables):

```
2 x = y = z = "Orange"
3 print(x)
4 print(y)
5 print(z)
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Orange
Orange
Orange
```

- تفريغ مجموعة (Unpack a Collection) : إذا كان لدينا مجموعة من القيم داخل list مثلاً، فإن بايثون تسمح باستخراج القيم إلى متغيرات.

```
2 fruits = ["apple", "banana", "cherry"]
3 x, y, z = fruits
4 print(x)
5 print(y)
6 print(z)
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
apple
banana
cherry
```

• إدخال المستخدم input() :



- تستخدم لإدخال بيانات من قبل المستخدم.
- الدخل يُعامل دائماً كسلسلة نصية (string) ، حتى لو كان المدخل رقماً.
- لتحويل الناتج من سلسلة نصية إلى رقم (مثلاً)، يجب استخدام توابع التحويل مثل int()

↪ أمثلة :

```
2 x = input()
3 print("Type of x is : " , type(x))
4 y = int(input())
5 print("Type of y is : " , type(y))
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
5
Type of x is : <class 'str'>
5
Type of y is : <class 'int'>
```

```
2 x = input("Number is :")
3 print(x - 5)
4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Number is :10
Traceback (most recent call last):
  File "c:\vs projects\python projects\test.py", line 3, in <module>
    print(x - 5)
      ~~~~
TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

الشرح ظهر لدينا error لان التابع كما ذكرنا يرى كل شيء على أنه سلسلة نصية فلا يمكننا أن نطرح قيمة عددية من سلسلة نصية ولتلافي الغلط نقوم بتحويل الخرج إلى عدد صحيح مثلاً ثم نطرح

↪ مثال:

```
2 x = input("Number is :")
3 print(int(x) - 5)
4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Number is :10
5
```

كما نرى هنا لقد قمنا بتحويل الخرج حتى تكون العملية صحيحة .



```

2 x = "Hi" ; y = "Hello"
3 print("X is : " , x , " | " , "Y is : " , y)
4 y = x
5 x = "Nooo"
6 print("X is : " , x , " | " , "Y is : " , y)
7

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

X is : Hi | Y is : Hello
X is : Nooo | Y is : Hi

تقوم لغة بايثون بالتنفيذ سطرًا بسطر من الأعلى للأسفل، لذا يتم تغيير قيمة y الى x القديمة ولم تأخذ الجديدة.

معاملات المقارنة والمنطق (Operators)

• معاملات المقارنة: (Comparison Operators)

- == : المساواة.
- != : عدم المساواة.
- < : أصغر من.
- > : أكبر من.
- <= : أصغر أو تساوي.
- >= : أكبر أو تساوي.
- الخرج دائماً يكون True أو False
- مقارنة المحارف : عند مقارنة محرف بمحرف، تتم المقارنة بين قيمة هذا المحرف وقيمة ذاك المحرف، حيث تعتمد بايثون على تمثيل المحارف باستخدام ترميز ASCII لمعرفة القيمة. يمكن استخدام التابع ord() لمعرفة القيمة.

مثال :

```

2 #ex1
3 print("EX1 : " , 'a' > 'b')
4 #ex2
5 print("EX2 : " , 'ab' > 'bd')
6 #ex3
7 print("EX3 : " , ord('a' ))
8

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

EX1 : False
EX2 : False
EX3 : 97

- في المثال الثاني الخرج false لان قيمة المحرف d اكبر من b .

المعاملات المنطقية: (Logical Operators)

- and : يعطي True في حال كان الطرفان محققين.
- Or : يعطي True في حال كان إحدى أو كلا الطرفين محققين.
- not : عكس النتيجة.
- الأولوية Not ثم and ثم or .

• (Identity Operators) :

- Is يعطي True إذا كان كلا المتغيرين يشيران إلى نفس الكائن في الذاكرة.
- is not : يعطي True إذا كان كلا المتغيرين ليسا من نفس الكائن.

↪ مثال:

```
2 x = ["apple", "banana"]
3 y = ["apple", "banana"]
4 z = x
5
6 print(x is z)
7 print(x is y)
8 print(x == y)
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

True
False
True

▪ الفرق بين is و == :

- == (المساواة): تحقق مما إذا كانت قيم كلا المتغيرين متساوية.
- is : تحقق مما إذا كان كلا المتغيرين يشيران إلى نفس الكائن في الذاكرة.

هياكل البيانات (Collections) :

1. القائمة (List) :

• الخصائص :

- مجموعة مرتبة (ordered): العناصر لها ترتيب محدد وهذا الترتيب لن يتغير.
- قابلة للتغيير (changeable): يمكن تغيير العناصر وإزالتها داخل القائمة بعد إنشائها.
- تسمح بالقيم المكررة (allow duplicate values) : لأن القوائم تتميز بالفهرسة لذلك يمكن أن تحتوي على عناصر لها نفس القيمة .
- لا يشترط أن تكون العناصر من نفس النوع.
- فهرسة العناصر تبدأ من [0] .
- آخر عنصر يمكن الوصول إليه بـ index سالب مثل [-1] .

• توابع وإجراءات هامة:

- len(): لمعرفة عدد عناصر القائمة.
- append(value): لإضافة عنصر في نهاية القائمة.
- extend(list): لدمج قائمتين.
- clear(): لإفراغ القائمة (تبقى القائمة موجودة ولكن فارغة).
- pop(): لحذف آخر عنصر.
- pop(index): لحذف عنصر في فهرس معين.
- insert(index, value): للإضافة في مكان معين.
- del NameofList: لحذف كامل القائمة.
- del NameofList[index]: حذف عنصر معين .

✍ مثال

```

2 x = ["apple", 5, True]
3 y = ["banana", 1, 2]
4 z = x
5
6 print("X is : ", x)
7 print("Count of x is : ", len(x))
8 print("After append : ", x.append("nooo"), " New x is : ", x)
9 print("After extend : ", x.extend(y), " New x is : ", x)
10 print("After pop the last item : ", x.pop(), " New x is : ", x)
11 print("After pop the specified index : ", x.pop(1), " New x is : ", x)
12 #change value
13 x[0] = 'Hi'
14 print("New x is : ", x)
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

X is : ['apple', 5, True]
Count of x is : 3
After append : None New x is : ['apple', 5, True, 'nooo']
After extend : None New x is : ['apple', 5, True, 'nooo', 'banana', 1, 2]
After pop the last item : 2 New x is : ['apple', 5, True, 'nooo', 'banana', 1]
After pop the specified index : 5 New x is : ['apple', True, 'nooo', 'banana', 1]
New x is : ['Hi', True, 'nooo', 'banana', 1]

```

• التقطيع: (Slice) أهم ما يميز القوائم هي التقطيع وهي أخذ جزء من القائمة وتطبيق شيء ما عليه

$a[start: stop: step]$

مثلاً عكس القائمة $a[::-1]$ ✍ مثال : $a[2,3,5,7,99]$

- $a[:] = [2,3,5,7,99]$ كامل القائمة.
- $a[1:3] = [3,5]$ طباعة القائمة من الفهرس 1 إلى ما قبل الفهرس 3.
- $a[1:] = [3,5,7,99]$ طباعة القائمة من الفهرس 1 إلى الأخير
- $a[:3] = [2,3,5]$ طباعة القائمة من البداية إلى ما قبل الفهرس 3.
- $a[:-2] = [2,3,5]$ طباعة القائمة من البداية إلى ما قبل الفهرس -2 .
- $a[-1] = 99$ آخر عنصر
- ترتيب الفهارس : من البداية : من 0 الى 4 , من النهاية من -1 الى -5

- عملياً [start :stop:step] slice =
- [:-1] عكس القائمة

🔗 مثال :

```

~
2  x = ["apple", 5 , True , 2 , 7 , 22]
3  slice = x[0:4:2]
4  print(slice)
5  slice = x[::-1]
6  print(slice)
7

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

['apple', True]
[22, 7, 2, True, 5, 'apple']

```

• نسخ القوائم :

يتم تخزين مرجع (reference) من القائمة وليس نسخ للقائمة بحد ذاتها، ليكن لدينا x قائمة ما وقلنا $y=x$ أي أن y أصبح يشير إلى نفس القائمة التي يشير لها x .
لإجراء عملية نسخ (Copy) للقائمة، يتم كتابة $y = x[:]$.

2. (Dictionary) :

• الخصائص :

- مجموعة مرتبة، قابلة للتغيير، ولا تسمح بالتكرارات في المفاتيح (key) .
- العناصر هي أزواج من {key: Value} .
- القيمة (value) يمكن أن تكون أي شيء (مثل int أو string أو list)
- لإنشائه، يمكن استخدام {key: Value} . أو التابع dict() .

🔗 مثال :

```

2  #ex1
3  thisdict = {
4      "brand": "Ford",
5      "model": "Mustang",
6      "year": 1964
7  }
8  print("EX1 : ",thisdict)
9  #ex2
10 thisdict1 = dict(name = "John", age = 36, country = "Norway")
11 print("EX2 : ",thisdict1)
12

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

EX1 : {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
EX2 : {'name': 'John', 'age': 36, 'country': 'Norway'}

```

• الوصول للعناصر :

- عن طريق المفتاح (key) .
- عن طريق التابع (key).get()

```

2  #ex1
3  thisdict = {
4      "brand": "Ford",
5      "model": "Mustang",
6      "year": 1964
7  }
8  x = thisdict["model"]
9  print("Accessing the item by key : ",x)
10 #ex2
11 x = thisdict.get('model')
12 print("Accessing the item by get() : ",x)
13

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Accessing the item by key : Mustang
Accessing the item by get() : Mustang

• الوصول للمفاتيح والقيم والعناصر:

- keys(): للحصول على المفاتيح (داخل قائمة).
- values(): للحصول على القيم (داخل قائمة).
- items(): تقوم بإرجاع كل عنصر موجود داخل dict كـ tuple بداخل قائمة

مثال :

```

2  #EX1
3  car = {
4      "brand" : 'Ford',
5      "model" : 'Mustang',
6      "year" : 1964,
7  }
8  x = car.items()
9  print("Get items : " , x)
10 y = car.keys()
11 print("Get keys : " , y)
12 z = car.values()
13 print("Get values : " , z)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Get items : dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])
Get keys : dict_keys(['brand', 'model', 'year'])
Get values : dict_values(['Ford', 'Mustang', 1964])

• تعديل وإضافة العناصر:

- لتغيير عنصر عن طريق key او update({key: Value}).



مثال:

```

1  ~
2  #EX1
3  car = {
4      "brand" : 'Ford',
5      "model" : 'Mustang',
6      "year" : 1964,
7  }
8  x = car.items()
9  print("Get items : " , x)
10 y = car.keys()
11 print("Get keys : " , y)
12 z = car.values()
13 print("Get values : " , z)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Get items : dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])
 Get keys : dict_keys(['brand', 'model', 'year'])
 Get values : dict_values(['Ford', 'Mustang', 1964])

▪ إضافة عنصر Name_dic[key] = Value او .update()

حذف العناصر:

- pop(key): لحذف عنصر معين.
- popitem(): لحذف آخر عنصر.
- del Name_dic[key]: لحذف عنصر معين.
- del Name_dic: لحذف المجموعة كاملة.

مثال:

```

3  thisdict = {
4      "brand": "Ford",
5      "model": "Mustang",
6      "year": 1964
7  }
8
9  thisdict["color"] = 'red'
10 print('Adding an item by index key and assigning a value to it :', thisdict)
11
12 thisdict.update({"New": "New"})
13 print('Adding an item by update() :', thisdict)
14
15 thisdict.pop("New")
16 print('Remove the item with the specified key by pop :', thisdict)
17
18 del thisdict['color']
19 print('Remove the item with the specified key by del :', thisdict)
20
21 thisdict.popitem()
22 print('Remove the last item by popitem :', thisdict)
23
24 print('Final dictionary :', thisdict)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Adding an item by index key and assigning a value to it : {'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
 Adding an item by update() : {'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red', 'New': 'New'}
 Remove the item with the specified key by pop : {'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
 Remove the item with the specified key by del : {'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
 Remove the last item by popitem : {'brand': 'Ford', 'model': 'Mustang'}
 Final dictionary : {'brand': 'Ford', 'model': 'Mustang'}

3. (Tuple):

- الخصائص:
 - هي مجموعة مرتبة.
 - لا يمكن إجراء أي تغيير عليها (غير قابلة للتغيير).
 - يمكن إنشاؤها من خلال `x=()` او `x=tuple(( , , ))`.

4. المجموعة (Set): (فائدة)

- الخصائص:
 - هي مجموعة غير مرتبة (unordered).
 - غير قابلة للتغيير (unchangeable).
 - غير مفهرسة (unindexed).
 - لا تسمح بالتكرار.
 - انشاءها: `Name=set(( , , ))` او `Name={ , }`.

مثال:

```

2 #ex1
3 thisset = set(("apple", "banana", "cherry"))
4 print('with set() : ',thisset)
5 #ex2
6 set1 = {1, 5, 7, 9, 3}
7 print('Set1 is : ' , set1)
8

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

with set() : {'cherry', 'banana', 'apple'}
Set1 is : {1, 3, 5, 7, 9}

```

- ملاحظة هامة set: أسرع من list في عمليات البحث والإضافة والحذف.
- الوصول للعناصر: لا يمكننا الوصول إلى عناصر set لأنها غير مفهرسة وليس لديها key.
- المرور على العناصر: يمكن المرور على عناصرها باستخدام حلقة For.
- الاستعلام عن وجود قيمة: يتم باستخدام in مثل `print ("ban" in name of set )`.
- الإضافة: `add()`.
- إضافة مجموعة إلى مجموعة `update()`.
- تعريف مجموعة فارغة: إذا استخدمنا {}, فإن بايثون تعتبر هذا تعريف ل dict وليس set. لذلك يتم كتابة `x = set()`.
- الحذف: `remove()` , `discard()`.
- `del()` لحذف المجموعة كاملة.

```

1
2 #ex1
3 thisset = {"apple", "banana", "cherry"}
4 t1 = {1, 2, 3}
5 thisset.add("kiwi")
6 print("Add an item by add() : ", thisset)
7 thisset.update(t1)
8 print('Add items from another set by update() : ', thisset)
9 thisset.remove('banana')
10 print("Remove an item by remove(): ",thisset)
11 thisset.discard(1)
12 print("Remove an item by discard(): ",thisset)
13 #Delet all the set
14 del thisset
15

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Add an item by add() : {'apple', 'kiwi', 'banana', 'cherry'}
 Add items from another set by update() : {1, 2, 3, 'banana', 'apple', 'kiwi', 'cherry'}
 Remove an item by remove(): {1, 2, 3, 'apple', 'kiwi', 'cherry'}
 Remove an item by discard(): {2, 3, 'apple', 'kiwi', 'cherry'}

■ إضافة قائمة ل Set : يمكن إضافة أي شيء إلى set وحتى لو كانت list .

تكرار	قابلة للتغير	مرتبة	
✓	✓	✓	list
✓	✗	✓	tuple
✗	✗	✗	set
✗	✓	✓	dict

التحكم في سير البرنامج (Control Flow)

1. جملة الشرط (if statements) :

- يتم تنفيذ كتلة التعليمات البرمجية (block) إذا كان الشرط محققاً، وإذا كان غير محقق لا يتم التنفيذ.
- الترتيب: if ، ثم elif ، ثم else .



```

2  a = 200
3  b = 33
4  if b > a:
5      print("b is greater than a")
6  elif a == b:
7      print("a and b are equal")
8  else:
9      print("a is greater than b")
10

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

a is greater than b

عبارة pass :

لا يمكن أن تكون عبارات if فارغة، ولكن إذا كان لدينا block بدون if (لأي سبب)، نستخدم عبارة pass لتجنب ظهور خطأ (error).

pass هي null operation ، أي لا يحدث شيء عند تنفيذها.

مثال: if 50>40

Pass

2. حلقة التكرار (For statements) :

تستخدم للتكرار على تسلسل (سواء كان قائمة، سلسلة نصية، أو set).

الصيغة: for item in Collection: block.

دالة range() :

range(start: stop: step) . في حال عدم كتابة اي من القيم يتم استخدام القيم الافتراضية قيمة واحدة: تعتبر stop، وتكون البداية افتراضياً من الصفر، ويتوقف عند الرقم ما قبل المكتوب.

قيمتان: تعتبر start و stop، ويكون مقدار الخطوة افتراضياً 1.

تدعم التابع len() لمعرفة عدد العناصر بداخلها.

المرور على key في dict او القيم : for key in namedict

المرور على العناصر : for key,value in x.items()

3. بناء المجموعة الجديدة (Comprehensions)

بناء مجموعة جديدة من البيانات من مجموعة موجودة في سطر واحد فقط.

الصيغة العامة:

[if condition (اختياري) for item in collection العملية المطبقة على العنصر]


```

-
2  #ex1
3  r = [0 for i in range(5)]
4  print('R is : ' , r)
5  #ex2
6  x = [[0 for i in range(6)]
7        for x in range(5)]
8  print('X is : ' , x)
9
10

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

R is : [0, 0, 0, 0, 0]
X is : [[0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0]]

```

في المثال الأول تم إنشاء قائمة من خمسة عناصر بقيمة 0

في المثال الثاني تم إنشاء قائمة من 5 عناصر كل عنصر منها قائمة من 6 عناصر

الدوال والفئات (Functions and Classes)

1. الدوال (Functions) :

- يتم تعريف الدالة باستخدام الكلمة المفتاحية `def` متبوعة باسم الدالة والأقواس والتتابع.
- الدوال في بايثون هي `object` وتستطيع إرجاعها.
- يمكننا تعريف دالة داخل دالة أخرى.
- يمكن تمرير وسطاء (`parameter`) .
- إذا مررنا وسيطاً وأسندنا له القيمة `None`، تلغى شرط تمرير قيمة له عند الاستدعاء.
- لا يتم تنفيذ الدالة إلا عند استدعائها بشكل صريح.
- إذا كان لدينا وسيطان في الدالة وعند استدعائها مررنا 3 وسطاء، سيؤدي ذلك إلى خطأ (`error`) .

مثال :

```

2  #ex1
3  def hello():
4      print("Hello World!")
5  hello()
6  #ex2
7  def hello(object):
8      print("Hello " + object + "!")
9  hello("cat")
10 hello("cat",1) #error
11

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Hello World!
Hello cat!
Traceback (most recent call last):
  File "c:\vs projects\python projects\test.py", line 10, in <module>
    hello("cat",1)
    ~~~~~^~~~~~
TypeError: hello() takes 1 positional argument but 2 were given

```

2. الفئات (Classes) :

- يتم إنشاء الكلاس بواسطة الكلمة المفتاحية class .
- المُنشئ (Constructor) :
هو أهم جزء في الكلاس.
يتم استدعاء هذه الدالة تلقائياً بمجرد إنشاء كائن جديد من الكلاس.
تعريف المُنشئ يكون: `def __init__(self, name):`
`self.name=name`
- الوسيط self :
الوسيط الأول في أي دالة داخل الكلاس يكون دائماً self .
هذا الوسيط يمثل الكائن نفسه، ويسمح لك بالوصول إلى سمات ودوال ذلك الكائن.

✍ مثال:

```

2 class Man:
3     def __init__(self, name):
4         self.name = name
5         print("Initialized!")
6     def hello(self):
7         print("Hello " + self.name + "!")
8     def goodbye(self):
9         print("Good-bye " + self.name + "!")
10 m = Man("David")
11 m.hello()
12 m.goodbye()
13

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Initialized!
Hello David!
Good-bye David!

(Scope and Globals)

- النطاق (Scope) : لا يمكن استخدام متغير تم تعريفه داخل دالة (local) كمتغير شامل (global) .
- يمكننا استخدام متغير من نوع global بداخل func
- لإنشاء متغير global بداخل دالة يمكننا استخدام global keywords

✍ مثال:

```

2 def myfunc() :
3     global x
4     x = 'fantastic'
5
6 myfunc()
7 print("python is : " , x)
8

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

python is : fantastic

- عند استخدام متغير داخل دالة بنفس اسم المتغير من نوع global فإن المتغير بداخل الدالة يسمى local
- 🔗 مثال:

```

2  x = 'awesome'
3
4  def myfunc() :
5      x = 'fantastic'
6      print("python is : " , x)
7  myfunc()
8  print("python is : " , x)
9

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

python is : fantastic
python is : awesome

```

توابع السلاسل النصية (String Methods)

- upper(): لجعل الأحرف كلها كبيرة.
- lower(): لجعل الأحرف كلها صغيرة.
- capitalize(): لجعل الحرف الأول كبير وباقي الأحرف صغيرة.
- count(char): لمعرفة عدد مرات تكرار حرف معين.

🔗 مثال :

```

2  x = 'fantastic'
3  print("Uper is : " , x.upper())
4  y = "HELLO"
5  print("Lower is : " , y.lower())
6  z = 'fafi'
7  print("Count is : " , z.count('f'))
8  j = 'hello'
9  print("Count is : " , j.lower().count('ll'))
10 print("Capitalize is : " , j.capitalize())
11

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Uper is : FANTASTIC
Lower is : hello
Count is : 2
Count is : 1
Capitalize is : Hello

```

الشرح : في المثال الاول تم تحويل المحارف إلى كبيرة

في المثال الثاني إلى صغيرة

في الثالث كم مرة تم تكرار المحرف f

في الرابع كم مرة تم تكرار المحرفين ll تم وضع lower() لان المحرفين في الكلمة ll ويتم حساب تكرار ll لذلك تم

تحويل أولا المحارف إلى صغيرة للمراعاة

الاستثناءات (Exceptions)

- لمعالجة الاستثناءات، يتم استخدام التركيب try except finally .
- Try: الكود المراد اختباره.
- Except: تنفيذ ما بداخلها في حال ظهور استثناء دون إنهاء البرنامج.
- Finally: يتم تنفيذه سواء ظهر استثناء أم لم يظهر.
- من أجل استدعاء استثناء نستخدم كلمة raise وهي تكافئ throw في جافا .

↪ مثال:

```
2 try:
3     x=7/0
4 except Exception as e:
5     print("Something went wrong")
6     print("Exception is : ",e)
7 finally:
8     print("The 'try except' is finished")
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Something went wrong
Exception is : division by zero
The 'try except' is finished

مكتبة Numpy

عند تنفيذ الشبكة العصبونية يتم حساب arrays و matrices و فئة المصفوفات في Numpy(numpy.array) توفر العديد من الطرق المريحة (الدوال والوظائف) التي تستخدم لتطبيق الشبكة بدلا من كتابة الالف الأسطر من الكود البرمجي العادي في بايثون لإجراء عملية ضرب مصفوفات والتي ستكون بطيئة جدا توفر لك المكتبة دوال محسنة للغاية وجاهزة للقيام بهذه العمليات بكفاءة وسرعة فائقة هذه الطرق تجعل الكود أقصر وأسرع وأسهل في القراءة يجب استيراد المكتبة من خلال Import numpy as np :

- لإنشاء مصفوفة نستخدم np.array() تستقبل هذه الدالة list أو tuple ومن ثم يتم تحويل هذه القائمة (list) إلى مصفوفة خاصة ب numpy تسمى numpy.ndarray



```

1
2 import numpy as np
3
4 arr = np.array([1, 2, 3, 4, 5])
5
6 print(arr)
7 print(type(arr))
8

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
[1 2 3 4 5]
<class 'numpy.ndarray'>

```

يمكن إجراء عمليات رياضية على المصفوفات ولكن يجب الانتباه إلى أن تكون عدد العناصر نفسها وإلا سيحدث

error

مثال :

```

~
2 import numpy as np
3
4 x = np.array([1.0, 2.0, 3.0])
5 y = np.array([2.0, 4.0, 6.0])
6 a = x + y
7 b = x - y
8 d = x * y
9 f = x / y
10 print("a is : ", a) ; print("b is : ", b)
11 print("d is : ", d) ; print("f is : ", f)
12

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
a is : [3. 6. 9.]
b is : [-1. -2. -3.]
d is : [ 2.  8. 18.]
f is : [0.5 0.5 0.5]

```

أيضا تتوفر العمليات الحسابية لمصفوفة numpy وقيمة عديدة في هذه الحالة تجرى العمليات الحسابية بين كل عنصر من عناصر المصفوفة والقيمة العديدة هذه الميزة تسمى broadcasting

مثال :

```

~
2 import numpy as np
3
4 x = np.array([1.0, 2.0, 3.0])
5 a = x / 2.0
6 print(a)
7

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
[0.5 1.  1.5]

```

يمكن إنشاء مصفوفات بأي عدد من الأبعاد مثل المصفوفات الأحادية والثنائية

- تسمى المصفوفات الأحادية (ذات البعد الواحد) vector متجه
- تسمى المصفوفات الثنائية matrix
- نسمي المصفوفات ذات الأبعاد الثنائية أو الثلاثية أو أكثر tensor
- مثلا `x=np.array([1,2],[3,4])` مصفوفة ثنائية سطرين وعمودين
- لمعرفة أبعاد المصفوفة نستخدم : `.shape` أو `.ndim`
- لمعرفة نوع المصفوفة Data type of the array نستخدم `.dtype`

↪ مثال :

```
2 import numpy as np
3
4 A = np.array([[1, 2], [3, 4]])
5 print(A)
6 print("shape is : ", A.shape)
7 print("dtype is : ", A.dtype)
8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\vs projects\python projects> python test.py
[[1 2]
 [3 4]]
shape is : (2, 2)
dtype is : int64
```

↪ (مثال على broadcasting) :

```
-
2 import numpy as np
3
4 A = np.array([[1, 2], [3, 4]])
5 print(A)
6 print("New A is ", A*10)
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\vs projects\python projects> python test.py
[[1 2]
 [3 4]]
New A is [[10 20]
 [30 40]]
```

- نستخدم index للوصول الى العناصر
- الوصول إلى عنصر في مصفوفة أحادية `arr[0]`
- للوصول الى عنصر في السطر الأول والعمود الثاني `arr[0][1]`
- نستخدم حلقة for من أجل الوصول إلى كل عنصر موجود

مثال :

```

-
2 import numpy as np
3
4 X = np.array([[51, 55], [14, 19], [0, 4]])
5 print("x is :", X)
6 print("x[0] : ", X[0])
7 print("x[0][1] : ", X[0][1])
8 for row in X:
9     print(row)
10

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
[[51 55]
 [14 19]
 [ 0  4]]
x[0] : [51 55]
x[0][1] : 55
[51 55]
[14 19]
[0 4]

```

■ لتحويل المصفوفة إلى أحادية البعد نستخدم `.flatten()`.
وبعد ما تم التحويل يمكننا الاستعلام (الوصول) الى عدة عناصر

مثال :

```

2 import numpy as np
3
4 X = np.array([[51, 55], [14, 19], [0, 4]])
5 X = X.flatten()
6 print("EX1 :",X[np.array([0, 2, 4])])
7

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
EX1 : [51 14 0]

```

■ في المثال الثاني قمنا بإرجاع القيم ذات index 0 و 2 و 4
يمكننا استخراج قيم من مصفوفة تحقق شرط معين

مثال :

```

2 import numpy as np
3
4 X = np.array([[51, 55], [14, 19], [0, 4]])
5 X = X.flatten()
6 print("EX : ",X>15)
7 print("EX1 : ",X[X>15])
8

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
EX : [ True  True False  True False False]
EX1 : [51 55 19]

```

الشرح : في المثال الأول نقارن قيمة قيمة وتكون النتيجة true أو false
أما في المثال الثاني النتيجة هي القيم الذي حققت الشرط

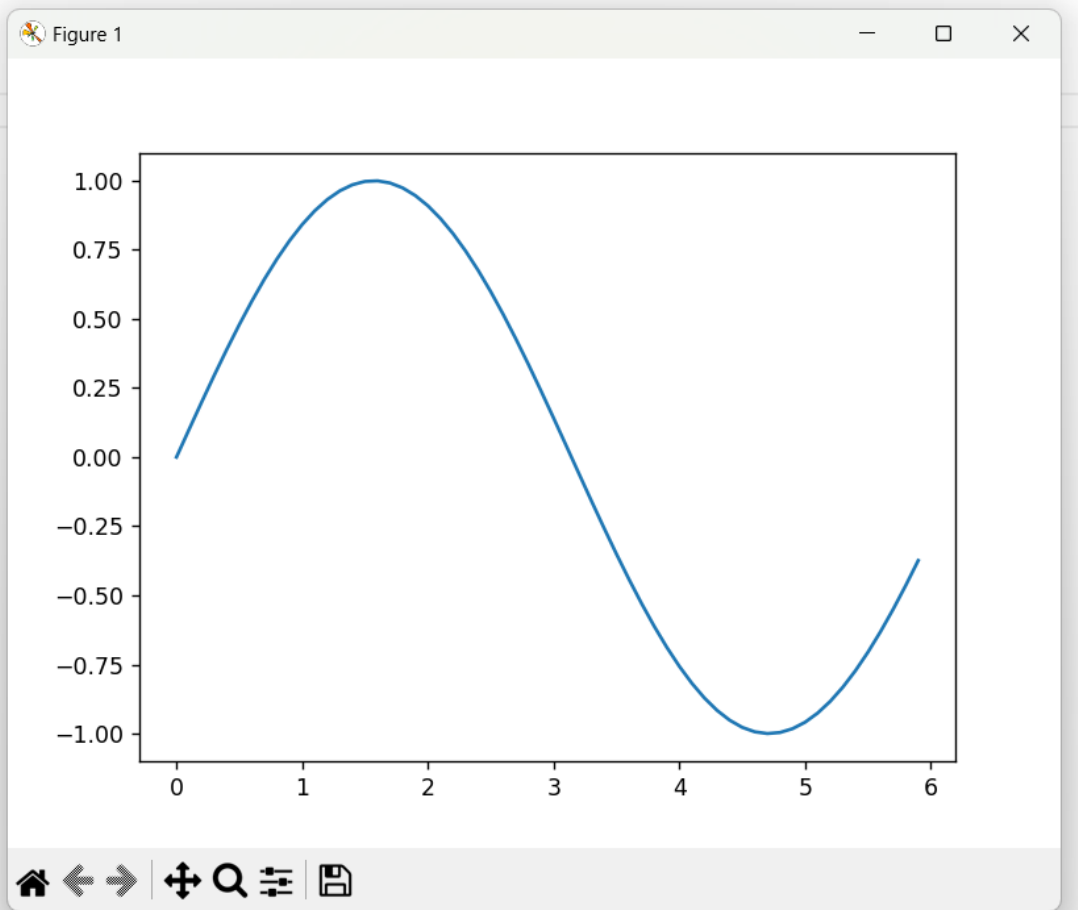
مكتبة matplotlib

يعد رسم الرسوم البيانية وتصور البيانات أمر مهم فمن خلال هذه المكتبة يمكننا الرسم بسهولة لأن matplotlib أداة مرنة وشاملة تجعل عملية تحويل الأرقام الناتجة عن عمليات numpy المعقدة إلى صور ومخططات سهلة وواضحة مما يسهل تحليل النتائج التجريبية

لا تقتصر المكتبة على المخططات الرياضية (مثل الخطوط والمنحنيات) بل يمكن استخدامها أيضا لعرض الصور وهذا أمر ضروري للتعامل مع مدخلات شبكات التعلم العميق مثل الصور (imshow())

🔗 مثال :

```
2 import numpy as np
3 import matplotlib.pyplot as plt
4 x = np.arange(0, 6, 0.1) # Generate from 0 to 6 in increments of 0.1
5 y = np.sin(x)
6 # Draw a graph
7 plt.plot(x, y)
8 plt.show()
9
10
11
12
13
```



1. يتم استخدام دالة `np.arange(0,6,0.1)` لتوليد بيانات المحور x تنشئ هذه الدالة مصفوفة تبدأ من 0 وتنتهي عند 6 ومقدار الخطوة هو 0.1 أي `[0,0.1,0.2.....]`
2. يتم تطبيق دالة `sin` على كل عنصر من عناصر المصفوفة x للحصول على قيم المحور y
3. الرسم والعرض : يتم تحرير مصفوفات x و y إلى دالة `plt.plot(x,y)`
4. لرسم المنحني يتم استدعاء الدالة `plt.show()` لعرض الرسم البياني على الشاشة

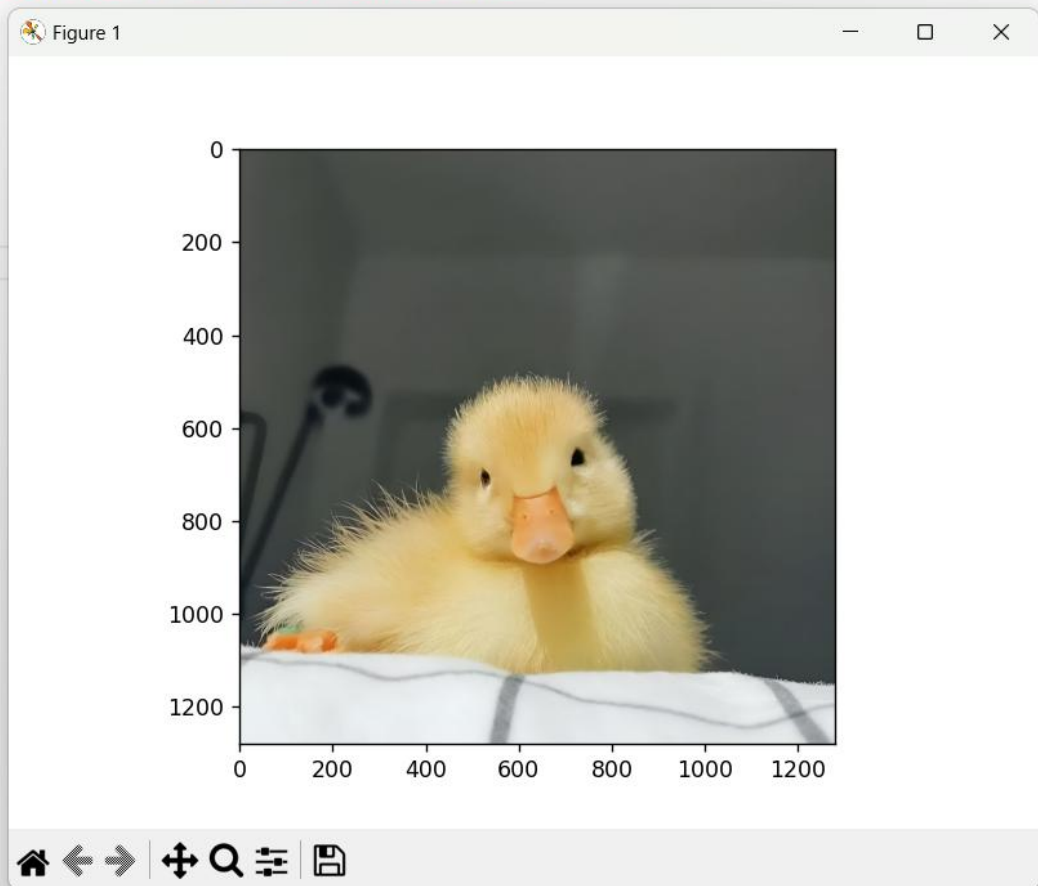
عرض الصورة :

مثال:

```

2 import matplotlib.pyplot as plt
3 from matplotlib.image import imread
4 path = r'C:\Users\ASUS\Pictures\Saved Pictures\duck.jpg'
5 img = imread(path)
6 plt.imshow(img)
7 plt.show()

```



1. لكي يتم عرض الصورة يجب أولاً قراءتها وتحميلها في الذاكرة كبيانات رقمية يتم ذلك لاستخدام الدالة `imread()` اختصار ل (image read)
 2. تقوم هذه الدالة بتحويل الصورة إلى مصفوفة `numpy` متعددة الأبعاد حيث يمثل كل رقم شدة لون (أو قيمة بكسل) معينة
 3. عرض الصورة : الدالة `plt.imshow()`
- بمجرد تحميل الصورة كبيانات رقمية في متغير تستخدم الدالة `imshow()` تعرض هذه المصفوفة ثنائية أو ثلاثية على أبعاد الشاشة كصورة مرئية

النهاية